

제02장.

- 오라클 접속에서 새로운 유저 생성

```
create user user02 identified by pass;  
grant connect, resource, dba to user02;
```

- USER02 접속 SQL 워크시트에서 출력 활성화

```
SET SERVEROUTPUT ON;
```

- SELECT문의 사용

- 1) 교수 테이블에서 교수 인원수와 교수 평균급여를 출력한다.

```
create or replace PROCEDURE EX01  
AS  
    v_count NUMBER;  
    v_avg_salary NUMBER;  
BEGIN  
    select count(*) into v_count from professors;  
    select avg(salary) into v_avg_salary from professors;  
    DBMS_OUTPUT.PUT_LINE('전체 교수 인원수:' || v_count);  
    DBMS_OUTPUT.PUT_LINE('전체 교수 평균급여:' || v_avg_salary);  
END EX01;
```

- 2) 학과를 입력 받아 해당 학과의 인원수와 평균 급여를 출력한다.

```
create or replace PROCEDURE EX02(i_dept VARCHAR2)  
AS  
    v_count NUMBER;  
    v_avg_salary NUMBER;  
BEGIN  
    select count(*) into v_count from professors where dept=i_dept;  
    select avg(salary) into v_avg_salary from professors where dept=i_dept;  
    DBMS_OUTPUT.PUT_LINE(i_dept || '과 교수 인원수:' || v_count);  
    DBMS_OUTPUT.PUT_LINE(i_dept || '과 교수 평균급여:' || v_avg_salary);  
END EX02;
```

- 3) 교수번호를 입력 받아 교수명과 교수급여를 출력한다.

```
create or replace PROCEDURE EX03(i_pcode in VARCHAR2)  
AS  
    v_pname professors.pname%TYPE;  
    v_salary professors.salary%TYPE;  
BEGIN  
    select pname, salary into v_pname, v_salary from professors where pcode=i_pcode;  
    DBMS_OUTPUT.PUT_LINE('교수명:' || v_pname);  
    DBMS_OUTPUT.PUT_LINE('교수급여:' || v_salary);  
EXCEPTION  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('오류발생:' || SQLERRM);  
END EX03;
```

4) %ROWTYPE을 사용하면 행 전체(모든 칼럼) 구조를 가져와서 변수 선언과 INTO 절을 훨씬 간단하게 할 수 있다.

```
create or replace PROCEDURE EX04(i_pcode in VARCHAR2) AS
    v_row professors%ROWTYPE;
BEGIN
    select * into v_row
        from professors where pcode=i_pcode;
    DBMS_OUTPUT.PUT_LINE('교수명:' || v_row.pname);
    DBMS_OUTPUT.PUT_LINE('교수급여:' || v_row.salary);
    DBMS_OUTPUT.PUT_LINE('교수학과:' || v_row.dept);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('오류발생:' || SQLERRM);
END EX04;
```

• INSERT문의 사용

1) %ROWTYPE 사용하여 레코드 변수를 사용하면 프로그램을 훨씬 간단하게 작성할 수 있다.

```
create or replace PROCEDURE EX05
AS
    v_row professors%ROWTYPE;
BEGIN
    v_row.pcode := '312';
    v_row.pname := '강감찬';
    v_row.dept := '전자';
    v_row.hiredate := SYSDATE;
    v_row.title := '조교수';
    --v_row.salary := 0;
    INSERT INTO professors VALUES v_row;
    DBMS_OUTPUT.PUT_LINE('INSERT 건수:' || SQL%ROWCOUNT);
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        IF SQLCODE = -1 THEN
            DBMS_OUTPUT.PUT_LINE('중복키 오류 발생');
        ELSE
            DBMS_OUTPUT.PUT_LINE('기타 오류:' || SQLERRM);
        END IF;
END EX05;
```

2) 교수 테이블에 '510', '성춘향' 데이터를 등록한다.

```
create or replace PROCEDURE EX06(
    i_pcode professors.pcode%TYPE,
    i_pname professors.pname%TYPE)
AS
BEGIN
    INSERT INTO professors(pcode, pname)
        VALUES (i_pcode, i_pname);
    DBMS_OUTPUT.PUT_LINE('INSERT 건수:' || SQL%ROWCOUNT);
    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('기타 오류:' || SQLERRM);
END EX06;
```

3) '전산', '홍길동'을 인자로 실행하면 전산과 가장 큰 코드 + 1이 새로운 코드로 등록된다.

```
create or replace PROCEDURE EX07(
    i_dept professors.dept%TYPE,
    i_pname professors.pname%TYPE)
AS
    v_pcode professors.pcode%TYPE;
BEGIN
    SELECT MAX(pcode)+1 INTO v_pcode FROM professors WHERE dept=i_dept;
    DBMS_OUTPUT.PUT_LINE('새로운 코드:' || v_pcode);
    INSERT INTO professors(pcode, pname, dept, hiredate) VALUES(v_pcode, i_pname, i_dept, SYSDATE);
    DBMS_OUTPUT.PUT_LINE('INSERT 건수:' || SQL%ROWCOUNT);
    COMMIT;
END EX07;
```

• UPDATE문의 사용

1) '229' 교수의 이름을 '김길동'으로 변경한다.

```
create or replace PROCEDURE EX08 AS
    v_pcode professors.pcode%TYPE := '229';
    v_pname professors.pname%TYPE := '김길동';
BEGIN
    UPDATE professors SET pname = v_pname WHERE pcode = v_pcode;
    DBMS_OUTPUT.PUT_LINE('UPDATE 건수:' || SQL%ROWCOUNT);
    COMMIT;
END EX08;
```

2) %ROWTYPE 레코드 변수를 사용하여 '229' 교수의 이름을 '홍길동', 급여를 0으로 수정한다.

```
create or replace PROCEDURE EX09 AS
    v_row professors%ROWTYPE;
BEGIN
    v_row.pcode := '229';
    SELECT * INTO v_row FROM professors WHERE pcode=v_row.pcode;
    v_row.pname := '홍길동';
    v_row.salary := 0;
    UPDATE professors SET ROW = v_row WHERE pcode = v_row.pcode;
    DBMS_OUTPUT.PUT_LINE('UPDATE 건수:' || SQL%ROWCOUNT);
    COMMIT;
END EX09;
```

3) 교수 테이블의 학과 데이터가 NULL이면 '미정'으로 변경한다.

```
create or replace PROCEDURE EX10 AS
    v_count NUMBER := 0;
BEGIN
    SELECT COUNT(*) INTO v_count FROM professors WHERE dept IS NULL;
    IF v_count = 0 THEN
        DBMS_OUTPUT.PUT_LINE('학과명이 NULL인 데이터가 없습니다.');
```

```
    ELSE
        UPDATE professors SET dept='미정' WHERE dept IS NULL;
        DBMS_OUTPUT.PUT_LINE('UPDATE 건수:' || SQL%ROWCOUNT);
    END IF;
    COMMIT;
END EX10;
```

• DELETE문의 사용

1) 교수 테이블에서 직급이 NULL인 데이터를 모두 삭제한다.

```
create or replace PROCEDURE EX11 AS
BEGIN
    DELETE FROM professors WHERE title IS NULL;
    DBMS_OUTPUT.PUT_LINE('DELETE 건수:' || SQL%ROWCOUNT);
    COMMIT;
END EX11;
```

2) 교수 테이블에서 교수 번호를 입력받아 일치하는 교수 정보를 삭제한다.

```
create or replace PROCEDURE EX12(i_pcode professors.pcode%TYPE)
AS
    v_count NUMBER;
BEGIN
    SELECT count(*) INTO v_count FROM professors WHERE pcode=i_pcode;
    IF v_count = 0 THEN
        DBMS_OUTPUT.PUT_LINE('삭제할 코드가 없습니다.');
```

```
    ELSE
        DELETE FROM professors WHERE pcode = i_pcode;
        DBMS_OUTPUT.PUT_LINE('DELETE 건수:' || SQL%ROWCOUNT);
    END IF;
    COMMIT;
END EX12;
```

• ROLLBACK

1) 작업이 하나라도 실패하면 전체 작업이 취소된다.

```
create or replace PROCEDURE EX13 AS
BEGIN
    INSERT INTO professors(pcode, pname) VALUES('229', '홍길동');
    INSERT INTO professors(pcode, pname) VALUES('228', '이순신');
    INSERT INTO professors(pcode, pname) VALUES('230', '강감찬');
    COMMIT; --모든 작업이 성공하면 확정
    DBMS_OUTPUT.PUT_LINE('처리가 완료되어 커밋 되었습니다.');
```

```
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK; --하나라도 실패하면 전체 취소
        DBMS_OUTPUT.PUT_LINE('오류 발생! 모든 작업이 롤백 되었습니다.' || ', 에러내용:' || SQLERRM);
END EX13;
```

2) 1번째 작업만 실행되고 나머지 작업들은 취소된다.

```
create or replace PROCEDURE EX14
AS
BEGIN
    INSERT INTO professors(pcode, pname) VALUES('229', '홍길동');
    SAVEPOINT sp1;
    INSERT INTO professors(pcode, pname) VALUES('230', '이순신');
    SAVEPOINT sp2;
    INSERT INTO professors(pcode, pname) VALUES('231', '강감찬');
    ROLLBACK TO sp1; -- 지점1 이후의 작업들만 취소된다.
    COMMIT; --결과적으로 홍길동만 등록되고 최종 확정된다.
END EX14;
```

• 컬렉션 타입(중첩 테이블)

오라클의 Nested Table(중첩 테이블)은 PL/SQL 메모리상에 가변 배열 형태로 관리하는 데이터 타입 또는 테이블 내의 한 칼럼이 또 다른 형태의 테이블(집합) 데이터를 가질 수 있도록 하는 객체 관계형 데이터 타입을 말한다. 일반적인 관계형 DB가 '1개 컬럼 = 1개 값'을 갖는 것과 달리, 하나의 행 안에 여러 개의 연관된 데이터를 집합 형태로 저장할 수 있다.

1) Nested Table과 FORALL을 조합하여 Nested Table에 저장한 교수코드에 해당하는 급여를 BULK UPDATE 예제이다.

```
create or replace PROCEDURE EX15 AS
  TYPE TYPE_CODE IS TABLE OF professors.pcode%TYPE;
  array_pcode TYPE_CODE := TYPE_CODE('221', '228');
BEGIN
  --1.컬렉션에 데이터가 있는지 확인
  IF array_pcode.COUNT > 0 THEN
    --2.BULK UPDATE 실행 (LOOP은 업데이트를 두 번 호출했겠지만 FORALL은 SQL 엔진에 업데이트 명령을 단 한 번에 던진다.)
    FORALL i IN 1..array_pcode.COUNT
      UPDATE professors SET salary = salary * 1.1 WHERE pcode = array_pcode(i);

    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || '명의 교수 급여가 인상되었습니다.');
```

--정상 실행한 경우 트랜잭션 확정

```
    COMMIT;
  END IF;
EXCEPTION
  WHEN OTHERS THEN
    --에러 발생한 경우 롤백
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('에러 발생: ' || SQLERRM);
END EX15;
```

2) professors 원본 데이터를 BULK로 한꺼번에 로딩 후 새로운 professors_history 테이블에 %ROWTYPE 변수를 통째로 삽입한다.

```
--교수 테이블을 복사할 비어있는 교수 History 테이블을 생성한다.
create table professors_history as
  select * from professors where pcode is null;
```

```
create or replace PROCEDURE EX16 AS
  --1.테이블 구조를 그대로 담은 컬렉션 타입 정의
  TYPE TYPE_PROFESSORS IS TABLE OF professors%ROWTYPE;
  array_professors TYPE_PROFESSORS;
BEGIN
  --2.BULK COLLECT로 원본 데이터를 메모리에 한꺼번에 로드
  SELECT * BULK COLLECT INTO array_professors FROM professors;
  --3. FORALL을 이용한 초고속 일괄 INSERT
  FORALL i IN 1..array_professors.COUNT
    --%ROWTYPE 변수를 통째로 삽입 가능
    INSERT INTO professors_history VALUES array_professors(i);

  DBMS_OUTPUT.PUT_LINE(array_professors.COUNT || '건의 데이터가 한꺼번에 저장되었습니다.');
```

--정상 실행한 경우 트랜잭션 확정

```
  COMMIT;
EXCEPTION
  WHEN OTHERS THEN
    --에러 발생한 경우 롤백
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('에러 발생: ' || SQLERRM);
END EX16;
```

• 커서(CURSOR)

오라클 커서(Cursor)는 SQL 문장(특히 SELECT 쿼리) 실행 결과로 반환된 여러 행(Row)의 집합을 메모리 영역(Context Area)에서 가리키는 포인터(Pointer)이다. 쿼리 결과에 순차적으로 접근하여 한 행씩 처리(Fetch)할 수 있다.

1) 교수 테이블에서 특정학과 교수들의 모든 정보를 출력하고 인원수를 출력한다.

```
create or replace PROCEDURE EX17(
    i_dept professors.dept%TYPE)
AS
    v_pcode professors.pcode%TYPE;
    v_pname professors.pname%TYPE;
    v_dept professors.dept%TYPE;
    v_title professors.title%TYPE;
    v_salary professors.salary%TYPE;
    --1.커서를 선언한다.
    CURSOR cur_professors IS
        SELECT pcode, pname, dept, title, salary FROM professors
            WHERE dept LIKE '%' || i_dept || '%';
BEGIN
    --2.커서를 OPEN한다.
    OPEN cur_professors;
    --3.커서 결과를 반복 FETCH한다.
    LOOP
        FETCH cur_professors INTO v_pcode, v_pname, v_dept, v_title, v_salary;
        EXIT WHEN cur_professors%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_pcode || ',' || v_pname || ',' || v_dept || ',' || v_title || ',' || v_salary);
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('전체인원:' || cur_professors%ROWCOUNT);
    --4.커서를 CLOSE한다.
    CLOSE cur_professors;
EXCEPTION
    WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('에러 발생:' || SQLERRM);
END EX17;
```

2) 위 예제를 커서에 대한 레코드 변수를 선언하여 FETCH한 결과를 레코드 변수에 저장 후 출력한다.

```
create or replace PROCEDURE EX18(
    i_dept professors.dept%TYPE)
AS
    CURSOR cur_professors IS
        SELECT * FROM professors WHERE dept LIKE '%' || i_dept || '%';
    --커서에 대한 레코드 변수 선언
    v_row cur_professors%ROWTYPE;
BEGIN
    OPEN cur_professors;
    LOOP
        --레코드 변수(v_row)에 결과를 저장한다.
        FETCH cur_professors INTO v_row;
        EXIT WHEN cur_professors%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_row.pcode || ',' || v_row.pname || ',' || v_row.dept || ',' || v_row.title || ',' || v_row.salary);
    END LOOP;
    DBMS_OUTPUT.PUT_LINE(i_dept || '과 인원:' || cur_professors%ROWCOUNT);
    CLOSE cur_professors;
EXCEPTION
    WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('에러 발생:' || SQLERRM);
END EX18;
```

- 커서 FOR LOOP

CURSOR FOR LOOP는 명시적 커서의 복잡한 과정(OPEN, FETCH, CLOSE)을 오라클이 자동으로 처리해주는 아주 편리하고 세련된 방식이다. 코드가 간결해지고 가독성이 높아져 실무에서 가장 선호한다.

1) 교수 테이블의 교수이름, 입사일자, 급여 목록을 출력한 후 급여 총합계를 출력한다.

```
create or replace PROCEDURE EX19 AS
  v_total_salary NUMBER := 0;
  --CURSOR cur_professors IS SELECT * FROM professors;
BEGIN
  FOR rec IN (SELECT * FROM professors) LOOP
    DBMS_OUTPUT.PUT_LINE(rec.pname ||
      ', 입사일자:' || TO_CHAR(rec.hiredate, 'YYYY-MM-DD') ||
      ', 급여:' || TO_CHAR(NVL(rec.salary, 0), '99,999,999'));
    v_total_salary := v_total_salary + NVL(rec.salary,0);
  END LOOP;
  DBMS_OUTPUT.PUT_LINE('-----');
  DBMS_OUTPUT.PUT_LINE('급여합계:' || TO_CHAR(v_total_salary, '99,999,999'));
END EX19;
```

2) 교수 테이블에서 교수들의 급여를 정교수인 경우 500, 부교수인 경우 100을 인상한다.

```
create or replace PROCEDURE EX20 AS
  CURSOR cur_professors IS SELECT * FROM professors;
  v_increase NUMBER := 0;
BEGIN
  FOR rec IN cur_professors LOOP
    IF (rec.title = '정교수') THEN
      v_increase := 500;
    ELSIF(rec.title = '부교수') THEN
      v_increase := 100;
    ELSE
      v_increase := 0;
    END IF;
    UPDATE professors SET salary=salary + v_increase WHERE pcode=rec.pcode;
  END LOOP;
END EX20;
```

3) 교수 테이블의 교수별 지도학생 목록과 지도학생 수를 출력한다.

```
CREATE OR REPLACE PROCEDURE EX21 AS
  CURSOR cur_professors IS
    SELECT * FROM professors ORDER BY pname;
  v_count NUMBER;
BEGIN
  FOR rec IN cur_professors LOOP
    DBMS_OUTPUT.PUT_LINE('교수번호:' || rec.pcode || ', 교수이름:' || rec.pname);
    v_count := 0;
    FOR rec2 IN (SELECT * FROM students WHERE advisor=rec.pcode) LOOP
      DBMS_OUTPUT.PUT_LINE('학생번호:' || rec2.scode || ', 학생이름:' || rec2.sname);
      v_count := v_count + 1;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('지도학생:' || v_count || '명');
    DBMS_OUTPUT.PUT_LINE('-----');
  END LOOP;
END EX21;
```

4) 학생 테이블의 학생 번호를 입력하면 해당 학생이 수강신청 한 강좌번호와 강좌이름을 출력한다.

```
CREATE OR REPLACE PROCEDURE EX22(i_scode students.scode%TYPE) AS
    v_lname courses.lname%TYPE;
    v_count NUMBER :=0;
BEGIN
    SELECT COUNT(*) INTO v_count FROM students WHERE scode=i_scode;
    IF v_count = 0 THEN
        DBMS_OUTPUT.PUT_LINE('해당 학생이 존재하지 않습니다.');
```

```
    ELSE
        FOR rec IN (SELECT * FROM enrollments WHERE scode=i_scode) LOOP
            SELECT lname INTO v_lname FROM courses WHERE lcode=rec.lcode;
            DBMS_OUTPUT.PUT_LINE('강좌코드:' || rec.lcode || ',강좌이름:' || v_lname);
        END LOOP;
    END IF;
END EX22;
```

5) 강좌 테이블의 강좌 번호를 입력하면 해당 강좌를 수강신청 한 학생번호와 학생이름을 출력한다.

```
CREATE OR REPLACE PROCEDURE EX23(i_lcode courses.lcode%TYPE) AS
    v_count NUMBER := 0;
    v_lcode courses.lcode%TYPE := UPPER(i_lcode);
    v_sname courses.lname%TYPE;
BEGIN
    SELECT count(*) INTO v_count FROM courses WHERE lcode = v_lcode;
    IF v_count = 0 THEN
        DBMS_OUTPUT.PUT_LINE('해당 강좌가 존재하지 않습니다.');
```

```
    ELSE
        FOR rec IN (SELECT * FROM enrollments WHERE lcode=v_lcode) LOOP
            SELECT sname INTO v_sname FROM students WHERE scode=rec.scode;
            DBMS_OUTPUT.PUT_LINE('학생번호:' || rec.scode || ', 학생이름:' || v_sname);
        END LOOP;
    END IF;
END EX23;
```

3) 강좌테이블의 강좌번호를 입력 받아 이 강좌를 신청한 학생들의 목록을 tbl_temp 테이블에 등록한다.

```
create table tbl_temp as select * from enrollments where lcode is null;
```

```
CREATE OR REPLACE PROCEDURE EX24(i_lcode courses.lcode%TYPE)
AUTHID CURRENT_USER --프로그램을 실행하는 사람의 권한 사용
AS
BEGIN
    BEGIN
        EXECUTE IMMEDIATE 'DROP TABLE tbl_temp';
    EXCEPTION
        WHEN OTHERS THEN
            IF SQLCODE = -942 THEN DBMS_OUTPUT.PUT_LINE('삭제할 테이블이 존재하지 않습니다.');
```

```
    END IF;
    END;
    EXECUTE IMMEDIATE 'CREATE TABLE tbl_temp AS SELECT * FROM enrollments WHERE lcode IS NULL';
    FOR rec IN (SELECT * FROM enrollments WHERE lcode=UPPER(i_lcode)) LOOP
        INSERT INTO tbl_temp VALUES rec;
    END LOOP;
END EX24;
```